Open/Closed зарчим - Domain Class Component ашиглан React-д хэрэгжүүлэх

Open/Closed зарчим нь **"Програм хангамжийн модулиуд өргөтгөлд нээлттэй, харин өөрчлөлтөд хаалттай байх ёстой."** гэсэн зарчмыг баримталдаг.
React-д энэ зарчмыг **Domain Class Component** ашиглан хэрхэн хэрэгжүүлэх жишээ

---

1. Open/Closed зарчмыг зөрчсөн буруу жишээ

Доорх код нь **User** классыг ашиглан хэрэглэгчийн мэдээллийг хадгалдаг. Гэхдээ шинэ хэрэглэгчийн төрөл (жишээ нь, **AdminUser**, **GuestUser**) нэмэх шаардлагатай бол **User классыг өөрчлөх** шаардлагатай болно.

```
jsx
CopyEdit
class User {
  constructor(name, role) {
    this.name = name;
    this.role = role;
  }

  getPermissions() {
    if (this.role === "admin") {
      return ["read", "write", "delete"];
    } else if (this.role === "user") {
      return ["read", "write"];
    }
    return ["read"];
  }
}

// Компонент
import React, { Component } from "react";

class UserComponent extends Component {
  render() {
    const { user } = this.props;
    return (
      <div>
        <h2>User: {user.name}</h2>
        <p>Role: {user.role}</p>
        <p>Permissions: {user.getPermissions().join(", ")}</p>
      </div>
    );
  }
}

export default UserComponent;
```

🚨 Яагаад энэ буруу вэ?

- Шинэ хэрэглэгчийн төрөл нэмэх (жишээ нь, **GuestUser**) үед `User` классыг **өөрчлөх** шаардлагатай болно.

- Энэ нь **Open/Closed зарчмыг зөрчиж байна**, учир нь код **өөрчлөлтөд нээлттэй** байгаа.

---

2. Open/Closed зарчмыг баримталсан зөв жишээ

Доорх код нь **User** классыг өөрчлөхгүйгээр өргөтгөх боломжтой байдлаар зохион байгуулагдсан.

User-ийг өвлүүлэн өргөтгөх (Шинэ хэрэглэгчийн төрлүүд нэмэх боломжтой)

```
jsx
CopyEdit
// Domain классууд
class User {
  constructor(name) {
    this.name = name;
  }

  getPermissions() {
    return ["read"];
  }
}

class AdminUser extends User {
  constructor(name) {
    super(name);
  }

  getPermissions() {
    return ["read", "write", "delete"];
  }
}

class RegularUser extends User {
  constructor(name) {
    super(name);
  }

  getPermissions() {
    return ["read", "write"];
  }
}
```

React компонент (Хэрэглэгчийн мэдээллийг харуулах)

```
jsx
CopyEdit
import React, { Component } from "react";

class UserComponent extends Component {
  render() {
    const { user } = this.props;
    return (
```

```
      <div>
        <h2>User: {user.name}</h2>
        <p>Permissions: {user.getPermissions().join(", ")}</p>
      </div>
    );
  }
}

export default UserComponent;
```

App.js (Компонентыг хэрэглэх)

```
jsx
CopyEdit
import React, { Component } from "react";
import UserComponent from "./UserComponent";
import { AdminUser, RegularUser } from "./User";

class App extends Component {
  render() {
    const admin = new AdminUser("Alice");
    const regularUser = new RegularUser("Bob");

    return (
      <div>
        <h1>Open/Closed зарчим - Domain Class Example</h1>
        <UserComponent user={admin} />
        <UserComponent user={regularUser} />
      </div>
    );
  }
}

export default App;
```

Яагаад энэ нь зөв шийдэл вэ?

✓ `User` классыг **өөрчлөхгүйгээр** шинэ төрлийн хэрэглэгч (AdminUser, RegularUser) нэмэх боломжтой.
✓ Код **өргөтгөлд нээлттэй, харин өөрчлөлтөд хаалттай** болсон.
✓ `UserComponent` классыг **өөрчлөхгүйгээр** шинэ хэрэглэгчийн төрлийг **шинэ классаар** хэрэгжүүлэх боломжтой.

---

3. Өөр нэг жишээ: Notification систем

Буруу жишээ (Open/Closed зарчмыг зөрчсөн)

```
jsx
CopyEdit
class Notification {
  constructor(type, message) {
    this.type = type;
```

```
    this.message = message;
  }

  getStyle() {
    if (this.type === "success") {
      return { backgroundColor: "green", color: "white", padding: "10px" };
    } else if (this.type === "error") {
      return { backgroundColor: "red", color: "white", padding: "10px" };
    }
    return { backgroundColor: "gray", color: "black", padding: "10px" };
  }
}
```

🚨 Асуудал:

- Шинэ төрлийн Notification (жишээ нь, `warning`) нэмэх үед `Notification` классыг өөрчлөх хэрэгтэй болно.

Зөв жишээ (Open/Closed зарчмыг дагасан)

```
jsx
CopyEdit
class Notification {
  constructor(message) {
    this.message = message;
  }

  getStyle() {
    return { backgroundColor: "gray", color: "black", padding: "10px" };
  }
}

class SuccessNotification extends Notification {
  getStyle() {
    return { backgroundColor: "green", color: "white", padding: "10px" };
  }
}

class ErrorNotification extends Notification {
  getStyle() {
    return { backgroundColor: "red", color: "white", padding: "10px" };
  }
}
```

React компонент

```
jsx
CopyEdit
import React, { Component } from "react";

class NotificationComponent extends Component {
  render() {
    const { notification } = this.props;
    return <div
style={notification.getStyle()}>{notification.message}</div>;
  }
```

```
}

export default NotificationComponent;
```

App.js

```
jsx
CopyEdit
import React, { Component } from "react";
import NotificationComponent from "./NotificationComponent";
import { SuccessNotification, ErrorNotification } from "./Notification";

class App extends Component {
  render() {
    const success = new SuccessNotification("Амжилттай хийгдлээ!");
    const error = new ErrorNotification("Алдаа гарлаа!");

    return (
      <div>
        <h1>Open/Closed зарчим - Notification</h1>
        <NotificationComponent notification={success} />
        <NotificationComponent notification={error} />
      </div>
    );
  }
}

export default App;
```

Дүгнэлт

1. **Хуучин кодыг өөрчлөхгүйгээр** шинэ хэрэглэгчийн төрөл эсвэл мэдэгдлийн төрөл нэмэх боломжтой.
2. **Open/Closed зарчмыг баримталсан** тул код илүү уян хатан болсон.
3. **Классийг өвлүүлэн өргөтгөх байдлаар** шийдсэн тул **хуучин кодыг өөрчлөхгүйгээр** шинэ төрлийн объект үүсгэх боломжтой.